

NestJS Document Parser POC

Technical Specification & Architecture Guide

1. Overview

This document parser POC demonstrates a production-quality NestJS application that compares multiple document parsing libraries by extracting text from uploaded files and presenting results side-by-side. The system is designed following Clean Architecture principles with strict TypeScript typing and full SOLID compliance.

2. Technology Stack

- Runtime: Node.js 24 with TypeScript 5.7 (strict mode)
- Framework: NestJS 11 with @nestjs/platform-express
- API Documentation: Swagger / OpenAPI 3.0 via @nestjs/swagger
- File Handling: Multer with disk storage (50 MB limit)
- Validation: class-validator + class-transformer
- Configuration: @nestjs/config with .env file support
- Logging: Winston with nest-winston integration
- Parser 1: markdown-js (PDF, DOCX, XLSX, PPTX, CSV, HTML, MD, TXT)
- Parser 2: officeparser (DOCX, PPTX, XLSX, PDF, ODS, ODP, ODT)

3. Architecture

The application follows a layered Clean Architecture. The DocumentParser interface acts as the primary abstraction contract. Each library is encapsulated in its own injectable service implementing getName(), supports(), getSupportedExtensions(), and parse(). The ParserFactory acts as a central registry, enabling the controller to remain decoupled from individual parser implementations.

4. API Endpoints

POST /parse/markitdown — Upload a file !' extract text via markitdown-js
POST /parse/officeparser — Upload a file !' extract text via officeparser
POST /parse/all — Upload a file !' run all supporting parsers in parallel
POST /benchmark — Upload a file !' run all parsers with timing + memory metrics
GET /supported-formats — Return file extensions supported per parser
GET /health — Service health check with timestamp

5. Benchmark Metrics

The /benchmark endpoint captures the following metrics for each parser run:

- durationMs — wall-clock time from parse start to completion
- charactersExtracted — total character count in extracted text
- words — whitespace-delimited word count
- lines — newline-delimited line count
- memoryBeforeBytes — Node.js heap usage before parsing
- memoryAfterBytes — Node.js heap usage after parsing

- `memoryUsedBytes` — net heap delta (after " before)

6. File Cleanup Strategy

Uploaded files are stored temporarily in `./uploads` using Multer's `diskStorage` with randomized filenames (timestamp + 8-byte hex). A `FileCleanupInterceptor` wraps every upload endpoint using `RxJS finalize()`, guaranteeing deletion after the response is sent regardless of success or error. This prevents disk accumulation without requiring a cron job.

7. Winston Logging

Every parse operation is logged at `INFO` level with context (parser name, file path, duration) and at `ERROR` level on failure with the full error message. In development mode, output is colorized in the NestJS nested format. In production, all transports emit structured JSON with timestamp, level, context, and message fields for log aggregation pipelines.

8. Extension Points

Adding a new parsing library requires only three steps: (1) create a new service class implementing the `DocumentParser` interface, (2) register it as a NestJS provider in `ParserModule`, and (3) inject it into the `ParserFactory` constructor. No changes to the controller, benchmark service, or any other component are required — a textbook Open/Closed principle implementation.

9. Security Considerations

- File type validation at both Multer `fileFilter` and NestJS Pipe levels
- Randomized upload filenames prevent path-guessing attacks
- Maximum file size enforced at 50 MB to prevent resource exhaustion
- Temp files auto-deleted after parsing to minimize attack surface
- No database — no SQL injection surface; fully stateless per request

10. Deployment Notes

The application requires no external services and is fully self-contained. The `/uploads` directory must be writable by the Node.js process. For containerized deployments, mount `/uploads` as a `tmpfs` volume for optimal cleanup speed. Set `NODE_ENV=production` to switch Winston to structured JSON output and disable color formatting.